27.08.25 Ex4:- Build a simple feed forward neural network to recognize handwritten characters

Aim:-
Build a simple neural network that can recognize handwritten characters.

Objective:-
1. Get the hand written image and labels.
2. Prepare the images so the computer can understand them.
3. Make a small neural network to learn patterns in the images.
4. Teach the network using many examples.
5. Check how well the network can recognize new images.

Pseudocode:-
START
Load training and test image
Prepare the images (flatten and normalize)
Create a neural network with:
  - Input layer (pixels)
  - one or two hidden layers.
  - output layer (number of characters)
Repeat for several times:
  Give a batch of images to the network
  Calculate how wrong the network is (loss)
  update the network to be better (train)

Test the network on new images
Show how accurate it is.

END

Output :-

Epoch 1 Accuracy : 90.29%.
Epoch 2 Accuracy : 95.57%.
Epoch 3 Accuracy : 97.06%.
Epoch 4 Accuracy : 97.70%.
Epoch 5 Accuracy : 98.50%.

Result :-

The feed forward neural network, trained on MNIST achieved around 95%. Approximately on the test data. This indicates the model learned to recognize most handwritten digits correctly. Although simple, it provides a solid baseline for digit classification task.

22/9

Accuracy :- $\frac{\text{True positive + true negative}}{\text{True positive + True negative + False positive + False negative}}$

F1 Score

F1 Score = $\frac{\text{True positive + True negative}}{\text{Total Instance}}$

Precision =

14.08.25 Exp:- 4:-

Build a simple feed forward neural network to recognize handwritten characters:-

Important terms:-

transforms.ToTensor() :- Convert images from PIL format (0-255 pixel values) to pytorch tensors (0-1. floats).

datasets.MNIST() : Loads the MNIST dataset.

train = True/False : Specifies training or test dataset.

DataLoader : Divides data into batches of size 32.

Shuffles training data for randomness; test data isn't shuffled.

nn.Module :- Base class for all neural networks.

1. Sigmoid function

$$f(x) = \frac{1}{1+e^{-x}}$$

- output range : (0,1)
- Pros : useful for probability outputs.
- cons : Causes vanishing gradients.

2. Hyperbolic Tangent (Tanh)

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

- output range : (-1,1).
- Pros : Stronger gradients than Sigmod.
- cons : Still suffers from vanishing gradient.

3. Rectified linear Unit (RELU)

$$f(x) = \max(0, x)$$

- output range : $[0, \infty)$
- Pros : fast, reduces vanishing gradient, widely used.
- cons : Dying ReLU problem.

4. Leaky ReLU:

$$f(x) = \begin{cases} x & x \geq 0 \\ 0.01x & x \leq 0 \end{cases}$$

fixed dying ReLU by allowing small negative slope.

5. Softmax

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Convert raw outputs into probability distributions.
- Commonly used in multi-class classification.